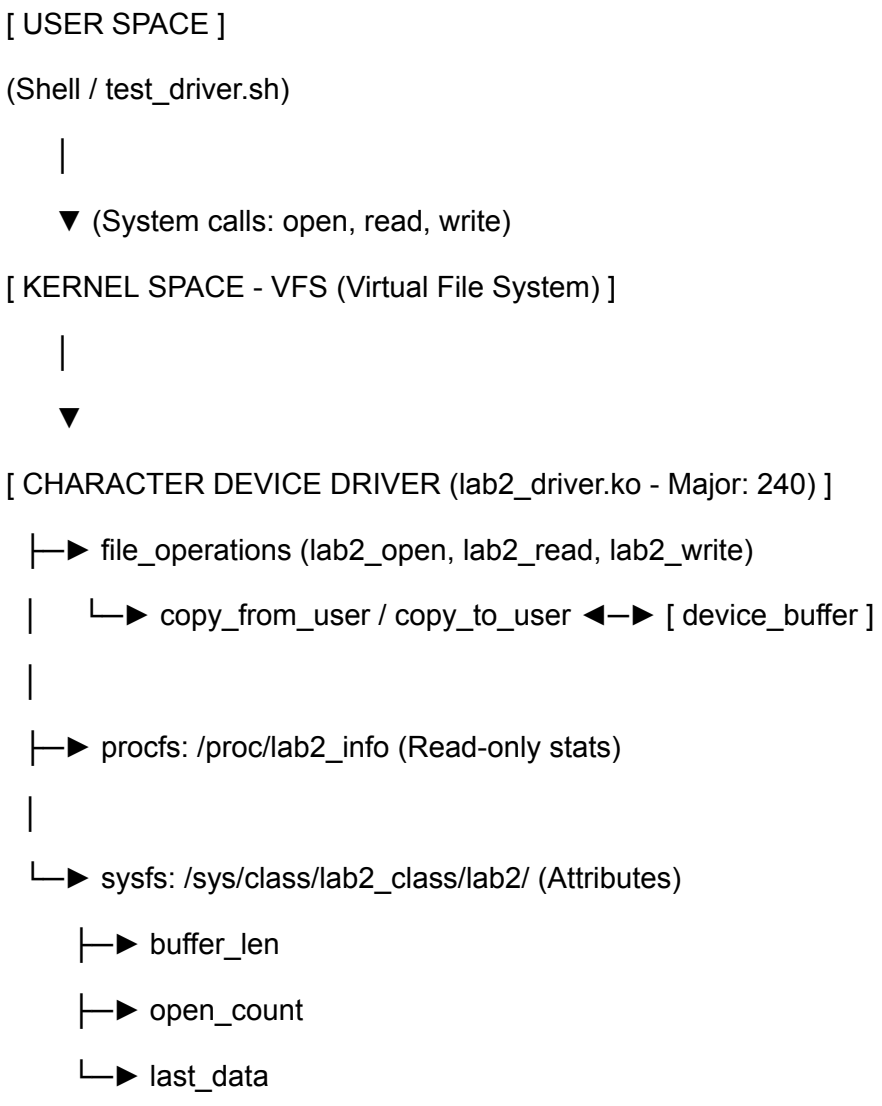
BÁO CÁO LAB-02: VIẾT DEVICE DRIVER & QUẢN LÝ FILE SYSTEM

MỤC 1: THÔNG TIN SINH VIÊN VÀ MÔI TRƯỜNG

- **Họ tên và MSSV:**
  - Hoàng Kiến Quốc - SE203373
  - Hoàng Minh Nhật - SE204273
- **Nhóm/Lớp:** Nhóm 1 - IC2009
- **Ngày thực hiện:** 20/10/2026
- **Phiên bản Kernel:** 5.15 LTS
- **Phiên bản GCC Toolchain:** GCC cross-compiler arm-linux-gnueabihf

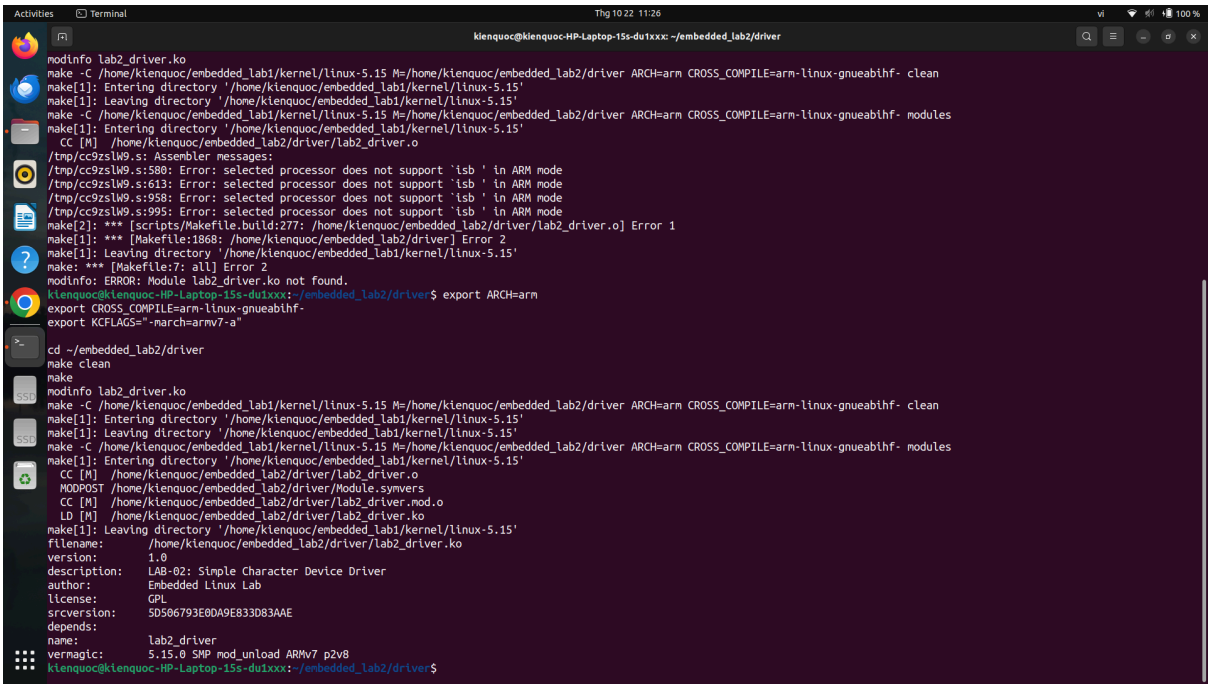
MỤC 2: SƠ ĐỒ KIẾN TRÚC DRIVER



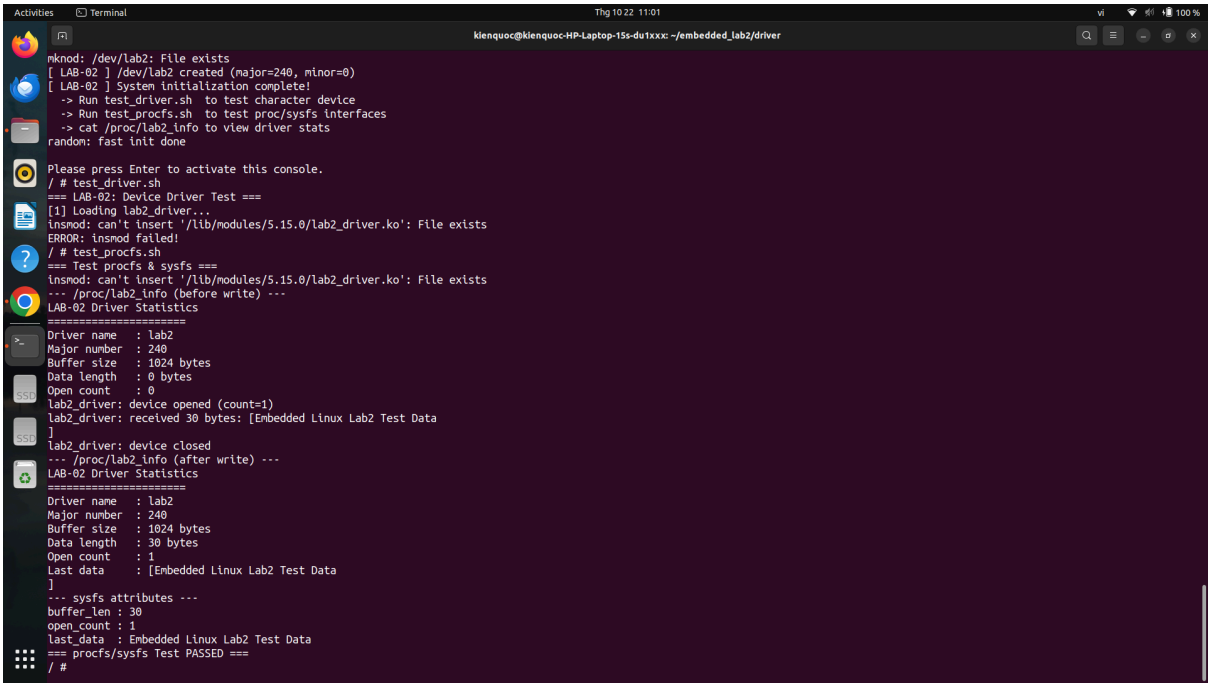
- **Luồng System Call:** Userspace program (gọi `open`, `read`, `write`, `close`) → VFS layer → `file_operations` trong `lab2_driver.ko` (Character device có major number 240).
- **Luồng Dữ liệu (Write):** User buffer → hàm `copy_from_user` → biến nội bộ `device_buffer` → hiển thị ra `procfs`/`sysfs`.
- **Kiến trúc Sysfs/Procfs:**
    - `/proc/lab2_info`: Báo cáo read-only dạng text về trạng thái driver.
    - `/sys/class/lab2_class/lab2/`: Chứa các attribute file quản lý biến tĩnh (`buffer_len`, `open_count`, `last_data`).

MỤC 3: LOG THỰC HÀNH

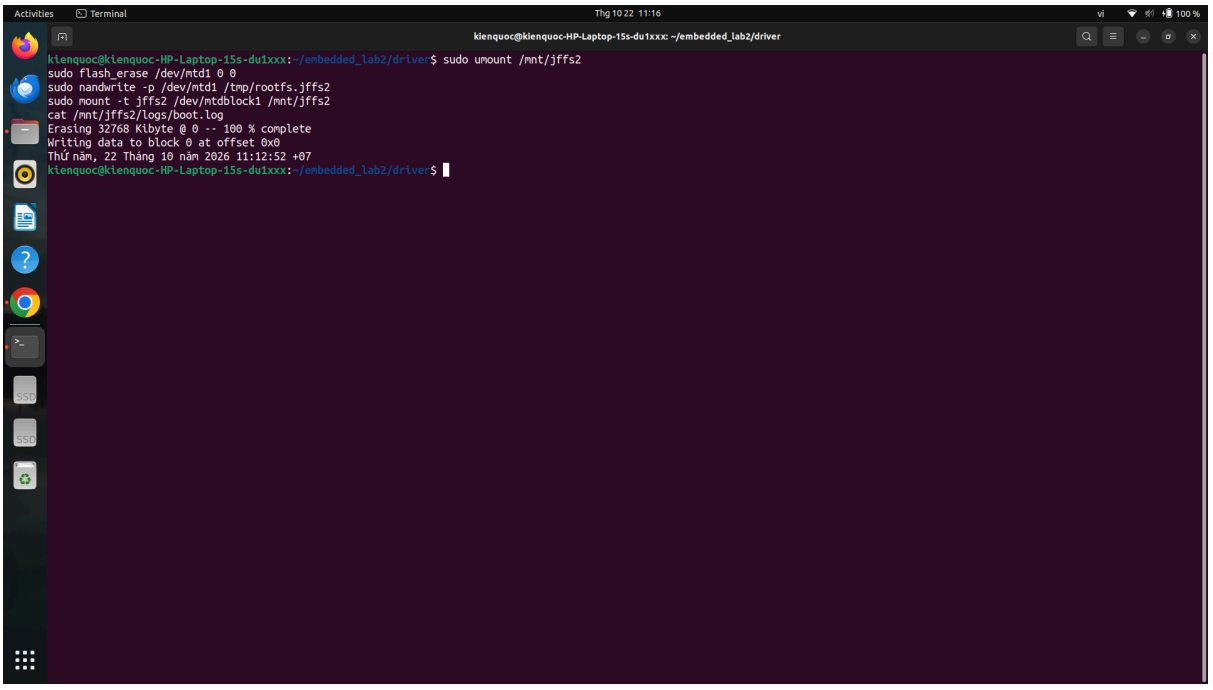
- **Phần A:** `[Chèn ảnh output lệnh modinfo, insmod, lsmod, kết quả đọc/ghi /dev/lab2 và dmesg | grep lab2]`



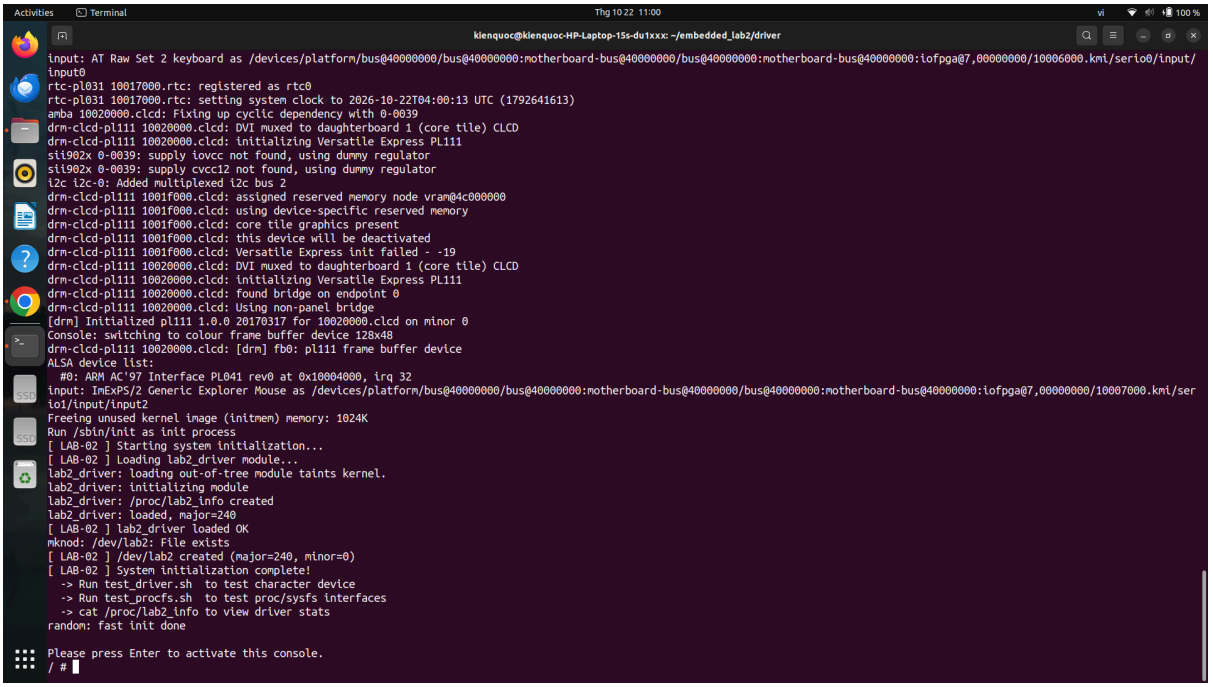
- **Phần B:** `[Chèn ảnh output cat /proc/lab2_info và toàn bộ các sysfs attributes]`



- **Phần C:** [Chèn ảnh output mtdinfo, flash_erase, nandwrite, mount JFFS2, đọc file, và kiểm tra tính bền vững (persistence) sau khi umount/mount]



- **Phần D:** [Chèn ảnh boot log QEMU hiển thị tự động load module, và output của test_driver.sh cùng test_procfs.sh]



MỤC 4: SO SÁNH FILESYSTEM

- **Bảng So sánh:**
  - **ext4:** Phù hợp cho thiết bị có block abstraction (SD card, eMMC); không phù hợp với Raw NAND/NOR do không tự quản lý wear leveling.
  - **JFFS2:** Tích hợp sẵn wear-leveling, bad block management, hỗ trợ nén, có thể đọc/ghi, lý tưởng cho thiết bị dùng Raw NAND/NOR. Tuy nhiên nó quét toàn bộ (full scan) lúc mount nên thời gian boot có thể chậm trên ổ quá lớn.
  - **cramfs:** Phù hợp để nén RootFS dạng Read-only (chỉ đọc), tiết kiệm bộ nhớ; không phù hợp để ghi hoặc thay đổi runtime.
- **Khuyến nghị:** Đối với thiết bị IoT có 32MB NAND flash cần cập nhật OTA thường xuyên, **JFFS2** là lựa chọn phù hợp nhất vì có tính năng read/write hỗ trợ ghi file cập nhật, đồng thời nó tự xử lý wear leveling bảo vệ độ bền của flash chip.

MỤC 5: TRẢ LỜI CÂU HỎI PHÂN TÍCH

- **Câu 1:** Kernel space và User space có vùng địa chỉ ảo (virtual address) riêng biệt. Không thể dùng hàm `memcpy()` trực tiếp vì kernel không thể truy cập trực tiếp và an toàn vào vùng nhớ của userspace. Thay vào đó, API `copy_to_user()` và `copy_from_user()` sẽ được sử dụng để truyền dữ liệu an toàn. Nếu cố tình dùng `memcpy()`, hệ thống có thể bị treo hoặc ghi đè bộ nhớ bất hợp pháp.
- **Câu 2:** Lệnh `DEFINE_MUTEX(lab2_mutex)` được sử dụng để ngăn chặn Race Condition khi nhiều tiến trình (process) cùng lúc thao tác trên biến dùng chung `device_buffer`. Nếu hai process cùng lúc gọi hàm `write()`, dữ liệu sẽ bị ghi đè chéo lên nhau hoặc làm sai lệch kết quả biến đếm, gây mất tính toàn vẹn dữ liệu.
- **Câu 3:** `procfs` chỉ phù hợp để cung cấp thông tin thống kê dạng text read-only (chỉ đọc). Trong khi đó, `sysfs` là một hệ thống có cấu trúc phân cấp, cho phép xem và điều chỉnh (read/write) từng thuộc tính riêng lẻ của driver thông qua kobject. Khuyến

nghị nên dùng `sysfs` để thiết lập tham số cho `buffer_len` hay `open_count` và dùng `procfs` hiển thị báo cáo.

- **Câu 4:** Cấu tạo vật lý của chip Flash NAND không cho phép ghi đè (overwrite); để đổi một bit từ 0 sang 1, ta bắt buộc phải xóa (erase) toàn bộ khối (erase block - thường là 128KB hoặc 256KB). File system JFFS2 khắc phục đặc tính này bằng cơ trúc log-structured, luôn ghi tuần tự vào block mới thay vì xóa ghi liên tục tại chỗ, đồng thời tự động phân bổ đều số lần xóa (wear leveling) để bảo vệ tuổi thọ phần cứng.
- **Câu 5:** BusyBox sẽ parse file `/etc/inittab` và chạy script `/etc/init.d/rcS`. Khi thêm lệnh `insmod` vào `rcS`, nếu module bị lỗi, việc script báo lỗi có làm treo tiến trình boot hay không phụ thuộc vào cách ta viết logic shell. Trong bài lab, rcS sử dụng khối lệnh `if - else` để bắt lỗi `insmod`; nếu thất bại, nó chỉ báo lỗi `ERROR` ra màn hình log nhưng không chặn hệ thống tiếp tục khởi động vào shell.

MỤC 6: NHẬN XÉT & KHÓ KHĂN

- **Khó khăn 1:** Gặp lỗi `insmod: ERROR: could not insert module`. **Cách khắc phục:** Nguyên nhân do chuỗi "vermagic" không khớp với kernel target. Đã build lại module trỏ đúng đường dẫn KDIR tới kernel source đã dùng ở LAB-01.
- **Khó khăn 2:** Khi chạy `mknod` báo lỗi `File exists`. **Cách khắc phục:** Do file device node đã tồn tại từ trước; phải gọi `rm /dev/lab2` để xóa trước, sau đó mới dùng `mknod` lại.